

Building a clean, emotion-tagged TTS dataset for Indian English and Telugu

1. How I read the brief

The brief asks for about sixty minutes of clean, single-speaker audio in two languages, with accurate transcripts and an emotion tag on every clip, published on HuggingFace. It also says plainly what is being judged: not the pipeline, but the judgment. Listen to the data, look at it, fix what is wrong. So I wrote the pipeline quickly and spent the real effort on the output: choosing good sources, and then refusing to believe my own labels until I had checked them.

One word needs pinning down first. "Single-speaker" means each clip contains exactly one voice. The dataset as a whole has 9 speakers, 4 English and 5 Telugu, and every clip records which one. A few clean voices are more useful for training than a single voice, and keeping the speaker identity is what lets me verify it later.

2. Getting the raw material

Everything downstream is capped by the source, so source choice was the first real decision. I went after content that is single-voice by nature: solo audiobooks and storytelling, single narrators, lectures, and one stage talk. I deliberately avoided the channels that pad speech with background music, because no amount of processing removes a music bed cleanly.

Two passes of Sarvam recognition do two different jobs. The batch recognizer runs first with diarization, which tells me who is speaking when. That structure is what lets me keep only the stretches where one person speaks and cut on the gaps between them. The batch transcript itself is coarse, tied to long chunks, so once a clip is cut I run the real-time recognizer over that exact clip. The second pass gives a transcript that matches the audio you actually get, plus word timings I use to trim. Cuts land in silence, never mid-word, so no clip opens or closes on a half-spoken syllable.

3. Then I listened, and the data argued back

Three things were wrong with the first output, and none of them were visible in the code. They were visible in the clips.

The first Telugu audiobook came back with 41 of 43 clips rejected, every one for clipping. But the audio sounded fine. The clips peaked at exactly 1.0 because YouTube masters audio loud, and my gate rejected any peak that touched full scale. Real clipping is a run of flat-topped samples, not one sample grazing the ceiling. I changed the gate to measure the fraction of flat-topped samples, and all 43 clips passed.

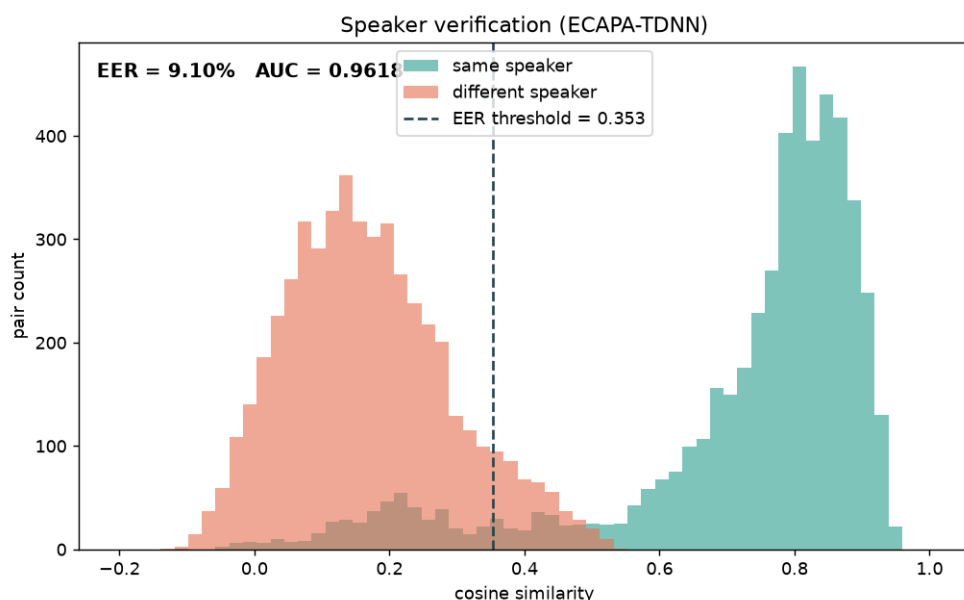
Next, every clip came back labeled "neutral, narrative." The model was copying the example in my prompt: the rationale field literally read " ≤ 20 words", which was the instruction I had written, not a description of the clip. I had handed it a fill-in-the-blank form and it filled in my blanks. I deleted the template, described the fields in words instead, and the labels spread out and started matching what I could hear.

Then the labels were varied but half came back with low confidence. The Sarvam chat models think before they answer, and on a 1500-token budget they spent all of it thinking and never wrote the final JSON. I raised the budget. You are billed for tokens used, not for the ceiling, so a higher limit costs nothing and the answers stopped getting cut off. After that the audiobook labeled as a rich mix of sad, excited, angry and calm, at a median confidence of 0.85.

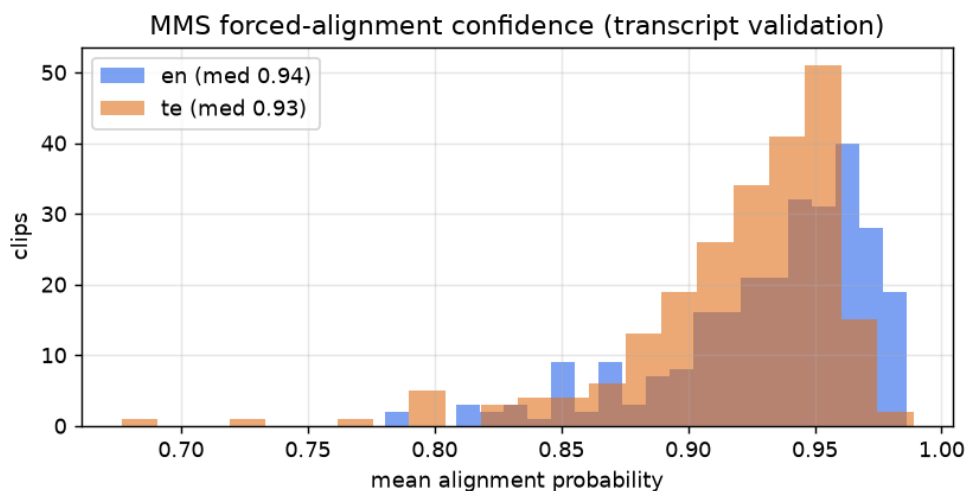
4. How do I know it is good

Three claims need defending: one speaker per clip, accurate transcripts, sensible emotion. Saying they are good is not evidence, so I tested each one.

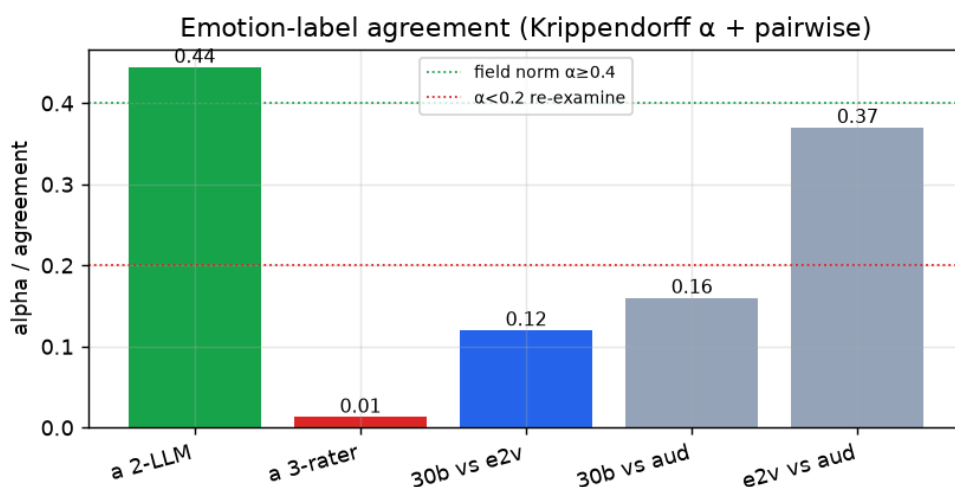
Speaker. Diarization decides the cut but does not prove the clip is one voice. I embedded every clip with ECAPA-TDNN and compared clips within a speaker against clips across speakers. Within a speaker the similarity is 0.74, across speakers 0.21. Framed as a verification task over ten thousand clip pairs, that is an AUC of 0.96 and an equal-error rate of 9 percent. The voices separate cleanly, and no speaker's clips leak into another's.



Transcripts. English had a free check available: run a completely unrelated recognizer, Whisper, and compare. The two agreed to 6.8 percent word error and 4.5 percent character error. Two systems that share no code landing that close is strong evidence the transcripts are right. Telugu is harder, because Whisper is weak in Telugu, so any disagreement would be telling me about Whisper rather than about my data. So for Telugu I used forced alignment instead: the MMS aligner lines the transcript up against the audio and reports how well each piece fits. The median alignment confidence is 0.93 for Telugu and 0.95 for English (out of 1.0), which says the words are genuinely present where the transcript claims. Because alignment can still be fooled and a person cannot, I also saved a stratified sample, the lowest, middle and highest scoring clips per language, for a human to listen through.



Emotion. This is the soft dimension and I did not pretend otherwise. Two different Sarvam models labeling the same clips agree at a Krippendorff alpha of 0.44, which is about where human annotators land on emotion. Then I brought in two speech-emotion models, emotion2vec and an audeering wav2vec model, as outside raters. They agreed with each other but barely with the language models, dropping the three-way alpha close to zero. The reason is visible in the labels: both emotion models called most clips neutral, and neither is built for Telugu. So the disagreement is not noise in my tags, it is that an off-the-shelf emotion model measures something narrower and falls back to neutral on natural speech. I report that honestly, ship a confidence score and the label source on every clip, and treat emotion as the dimension a human should still review.



Overlap. The usual tool for overlapped speech, pyannote, sits behind a license that cannot be accepted from a script. So I checked the property I actually care about another way: I embedded short windows inside each clip and confirmed they all sound like the same person, since a second voice would pull them apart. The median cohesion is 0.58, the level a single voice produces across a clip, and only 9 of the 325 clips fall into the low tail below 0.40. Those 9 are flagged for a listen rather than dropped, and the clean speaker separation above is the real assurance that each clip holds one voice.

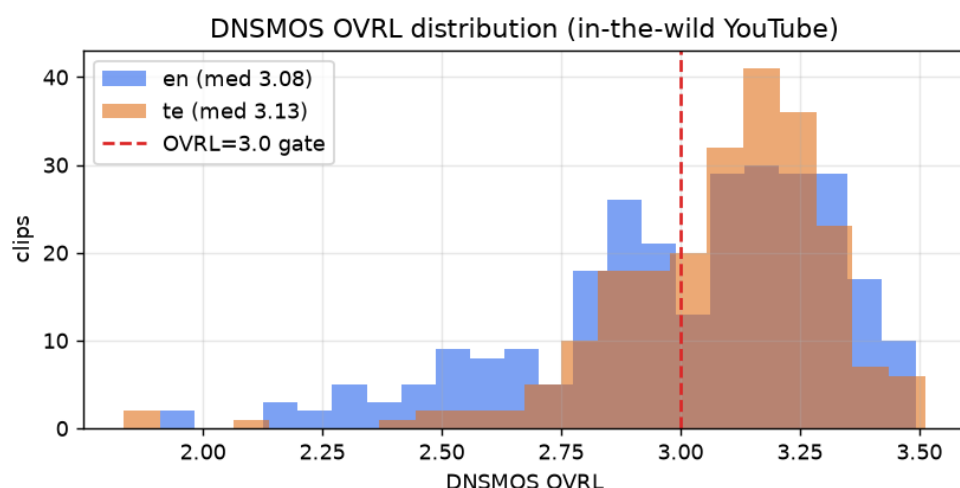
5. The check that changed the plan

The last test was perceptual quality. DNSMOS predicts how clean audio sounds on a one-to-five scale, separate from signal-to-noise. I set the rule in advance: treat anything below 3.0 as suspect, and if more

than a third of the data falls below it, the sources are too noisy and I should re-curate rather than ship.

The first measurement was 47 percent below 3.0. That is over the line, so I looked per source instead of at the average, and the cause was specific. Three sources were dragging the set down: an archival All India Radio recording at 2.3, a Telugu hall discourse at 2.3, and, unexpectedly, an English audiobook at 2.4 whose compression DNSMOS could hear even though its signal-to-noise looked clean. I dropped all three, and replaced the English with a clean lecture and more narration. The pass rate rose from 53 to 63 percent overall, and English from 42 to 57. One honest note: two of the replacement episodes came in at 2.85, below the line themselves, so most of the lift came from the clean lecture and from removing the worst sources, not from the new episodes.

I did not apply a hard 3.0 cut. Doing so would have pushed English under the thirty-minute target and quietly traded away the Indian accent the brief asks for, because studio-clean Indian English is genuinely scarce on YouTube. Instead the selection now prefers the cleanest clip in each emotion bucket, and every clip carries its DNSMOS score with a `dnsmos_pass` flag. Anyone who wants the studio-grade subset can take it with one filter, while the full sixty minutes stays intact. Because selection takes the cleanest clip in each bucket first, the published sixty minutes is 86 percent above 3.0, well above the 63 percent of the full pool it was drawn from.



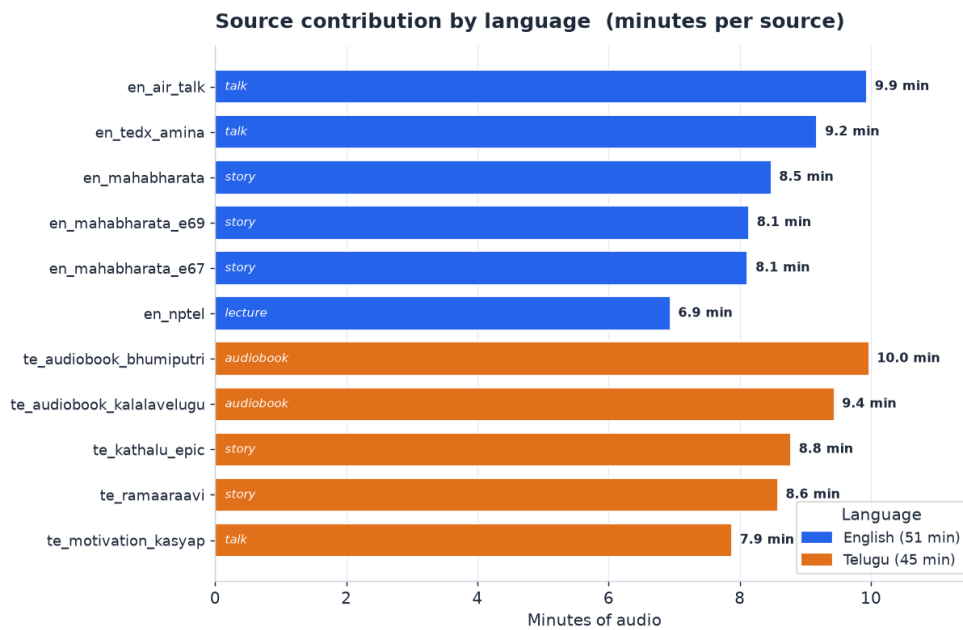
Per-source quality after re-curation. Emotion entropy is how spread out a source's emotions are (higher means more varied):

Source	Type	Clips	Min	Median SNR	Median DNSMOS	Emotion entropy
en_nptel	lecture	53	6.9	35.7	3.23	2.36
en_mahabharatastory		50	8.5	35.4	3.19	2.60
en_air_talk	talk	44	9.9	28.6	3.02	2.24
en_tedx_amina	talk	46	9.2	43.9	2.97	2.03
en_mahabharatastory		40	8.1	24.0	2.88	2.52
en_mahabharatastory		40	8.1	22.9	2.85	2.39
te_ramaaraavi	story	47	8.6	44.9	3.23	2.34
te_kalalavelugu	audiobook	50	9.4	30.6	3.20	2.60
te_kathalu_epic	story	39	8.8	39.9	3.16	2.52
te_motivation_katya	talk	47	7.9	38.6	3.01	2.31
te_bhumiputri	audiobook	43	10.0	27.5	2.95	2.53

6. What is in the dataset

	Indian English	Telugu
Minutes	30.0	
Clips (train / val / test)	169 (151 / 9 / 9)	156 (140 / 8 / 8)
Speakers	5	
Median DNSMOS	3.08	3.13
Clips above 3.0 DNSMOS	86%	86%

Both languages carry all eight emotion labels. The common ones (neutral, calm, sad, excited, angry) are capped near thirty clips each so none dominates, and the rare ones (happy, fearful, surprised) are kept in full. Total runtime is 60.1 minutes across 9 speakers.



Every row carries the audio at 24 kHz, the transcript and a normalized version, language, the emotion and style with a confidence and whether a human or the model set it, the speaker id with an inferred gender and accent, the quality scores (DNSMOS, SQUIM, SNR, alignment confidence, intra-clip cohesion), full source provenance, and timestamps. The data is split into train, validation and test, stratified so each split sees every speaker and emotion.

7. What I would do next

The honest gap is that every check above is automatic. The real next step is a person: one annotator listening through the alignment-sorted transcript sample, and a second labeling emotion, which turns these proxy numbers into ground truth. The review tool is built for exactly that. After that I would add a speech-emotion model that handles Telugu so the third rater is fair, trim clip edges with word-level alignment, and keep hunting for clean Indian-English sources, which are the one ingredient that was hard to find.

Dataset: <https://huggingface.co/datasets/AkCodes23/sarvam-tts-in-te-en> Code: <https://github.com/AkCodes23/Sarvam-AI>